Étude et optimisation de la détection de cycle

Algorithme Rho de Pollard et rupture du logarithme discret

Raphaël DEPUYDT — 50259

MPI

Session 2026

Le Contexte : L'échange de clés Diffie-Hellman

Alice et Bob veulent partager un secret S via un canal public.

1. Ils choisissent un grand premier p et un générateur g.
2. Alice choisit a (privé) et envoie $A = g^a \pmod{p}$.
3. Bob choisit b (privé) et envoie $B = g^b \pmod{p}$.
4. Le secret est $S = g^{ab} \pmod{p}$.

Le défi de l'attaquant

L'espion voit g, p, A et B. Pour calculer S, il doit trouver a ou b. C'est le problème du **Logarithme Discret (DLP)**.

Problématique

Question centrale

Comment l'algorithme Rho de Pollard permet de fragiliser les systèmes cryptographiques ?

Le Cadre Algébrique : $\mathbb{Z}/p\mathbb{Z}^*$

On travaille dans le groupe multiplicatif $G = (\mathbb{Z}/p\mathbb{Z})^*$.

- p : nombre premier (définit le corps de calcul).
- $q = p - 1$: ordre du groupe (définit le modulo des exposants).

Définition du Logarithme Discret (DLP)

Étant donnés g et $h \in G$, trouver l'unique $x \in \mathbb{Z}/q\mathbb{Z}$ tel que :

$$g^x \equiv h \pmod{p}$$

Exemple numérique

Trouver x tel que $2^x \equiv 5 \pmod{13}$.
$2^1 = 2, 2^2 = 4, 2^3 = 8, 2^4 = 3, \ldots, 2^9 = 512 \equiv 5 \pmod{13}$.
La solution est $x = 9$.

Complexité : Pourquoi Pollard ?

- **Recherche exhaustive :** On teste $x = 0, 1, 2 \ldots \rightarrow O(q)$.
- **Pollard Rho :** Utilise les collisions $\rightarrow O(\sqrt{q})$.

Analogie : Pour $q = 10^{22}$ (mille milliards) :

- Exhaustif : 10^{22} opérations (environ 3200 ans).
- Pollard : 10^{11} opérations (1 seconde).

Le Paradoxe des Anniversaires

Dans un ensemble de taille N, une collision apparaît avec probabilité $P = 1 - \frac{N!}{N^k(N-k)!}$, donc P > 0.5 après $k \approx 1.17\sqrt{N}$ tirages.

Idée de Pollard

Si on marche "au hasard" dans le groupe, on retombera sur un point déjà visité beaucoup plus vite qu'en parcourant tout le groupe.

Le Concept de l'Algorithme Rho

Le Principe : La recherche de collision

Au lieu de chercher directement l'exposant x (recherche exhaustive), on cherche une **collision** dans une suite aléatoire de nombres générés de manière itérative.

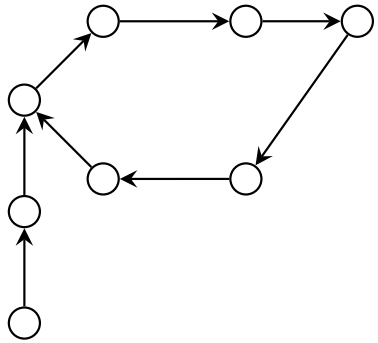
Complexité Espérée

D'après le paradoxe des anniversaires, la collision survient en moyenne après $\sqrt{q}$ itérations.

La Structure en Rho (ρ)

Graphes fonctionnels :

- M : Longueur de la queue (pré-période).
- L : Longueur du cycle.



Structure en ρ

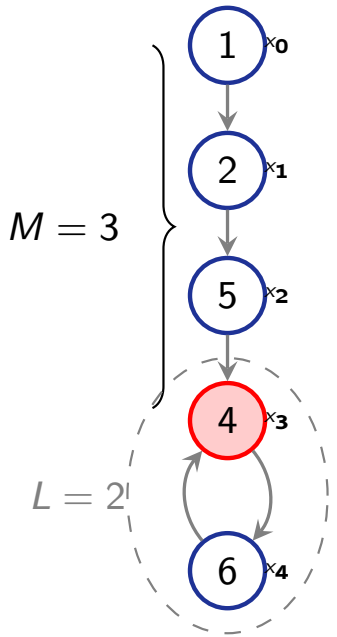
Exemple concret de structure en Rho

Paramètres :

- Groupe : $(\mathbb{Z}/11\mathbb{Z}, \times)$
- Fonction : $f(x) = x^2 + 1$
- Départ : $x_0 = 1$

Trace du calcul

$$x_0 = 1$$
$$x_1 = 1^2 + 1 = 2$$
$$x_2 = 2^2 + 1 = 5$$
$$x_3 = 5^2 + 1 \equiv 4$$
$$x_4 = 4^2 + 1 \equiv 6$$
$$x_5 = 6^2 + 1 \equiv 4 \quad \text{COLLISION}$$



Structure en ρ obtenue : une queue de 3 et un cycle de 2.

Statistiques de Flajolet-Odlyzko

D'après l'étude des graphes fonctionnels aléatoires de Flajolet-Odlyzko :

$$\mathbb{E}(M) \approx \mathbb{E}(L) \approx \sqrt{\frac{\pi q}{8}}$$

Conséquence théorique

L'espérance du temps de collision totale est $\mathbb{E}(M + L) \approx \sqrt{\frac{\pi q}{2}} \approx 1.25\sqrt{q}$.

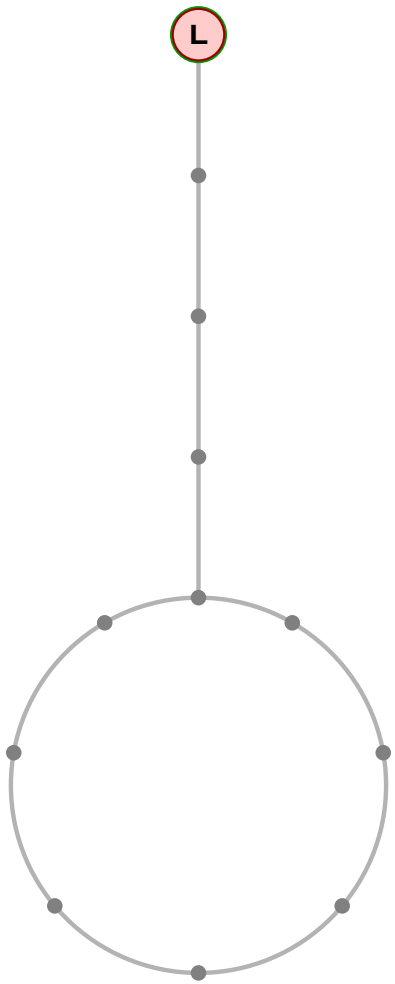
Algorithme de Floyd (Tortue et Lièvre)

Principe : "La Tortue et le Lièvre"

- Un pointeur x_i (lent) et un pointeur x_{2i} (rapide).
- Collision garantie dès que $i \geq M$ et $i \equiv 0 \pmod{L}$.

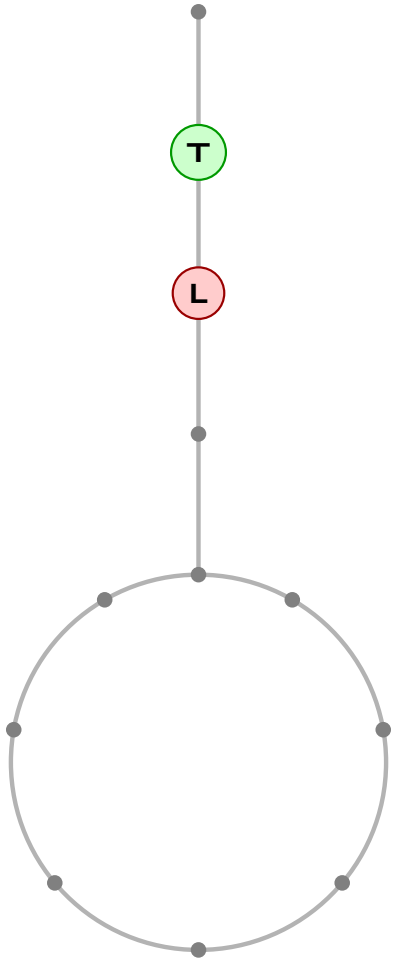
Évaluations de f / itération : 3
Espérence de la detection de cycle : $3.09\sqrt{q}$

Algorithme de Floyd (Tortue / Lièvre)



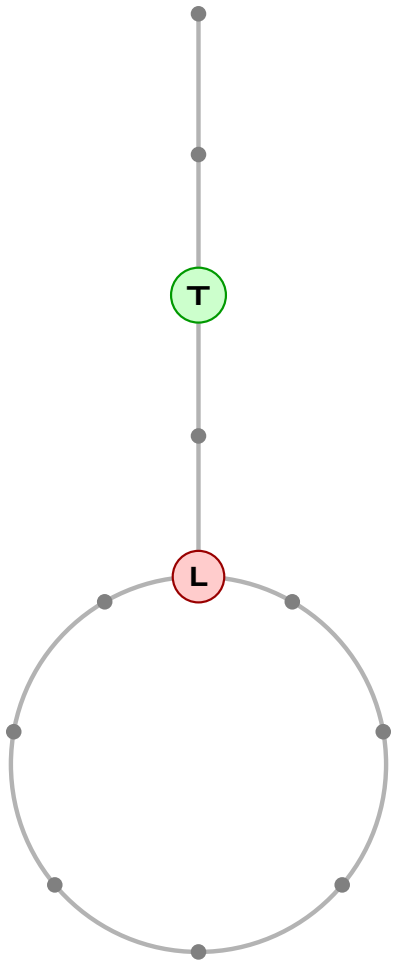
Étape 0 : $x_0 = x_{2(0)}$

Algorithme de Floyd (Tortue / Lièvre)



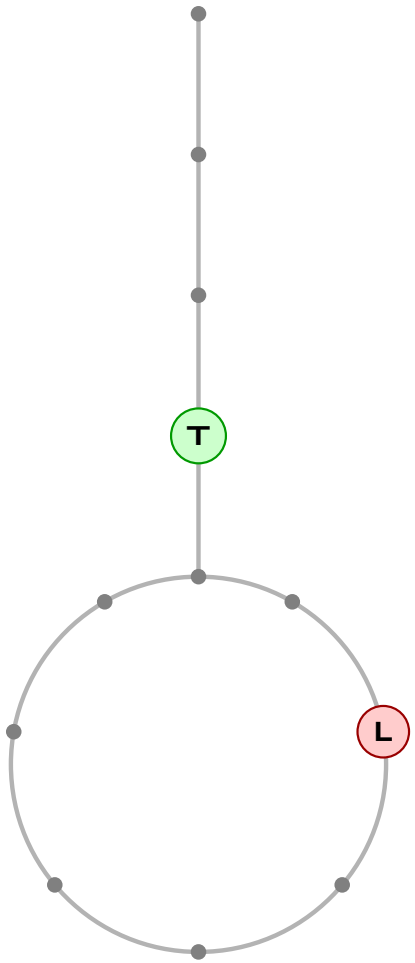
Étape 1 : Tortue=+1, Lièvre=+2

Algorithme de Floyd (Tortue / Lièvre)



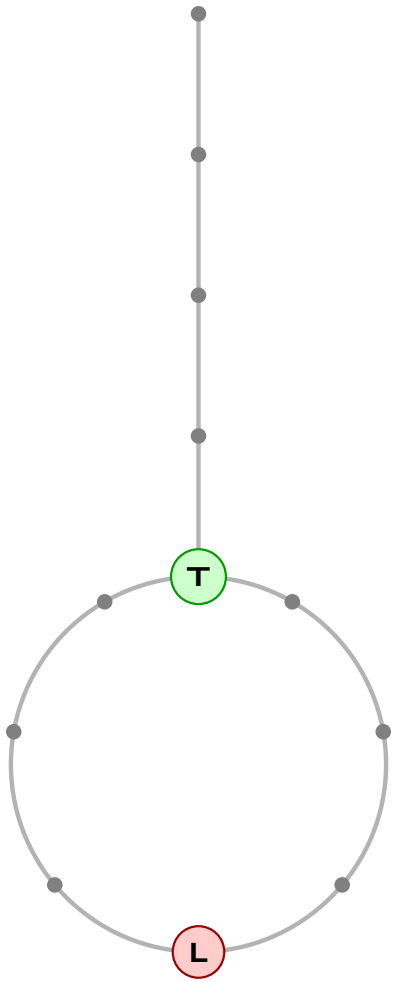
Étape 2 : Le lièvre atteint l'entrée du cycle

Algorithme de Floyd (Tortue / Lièvre)



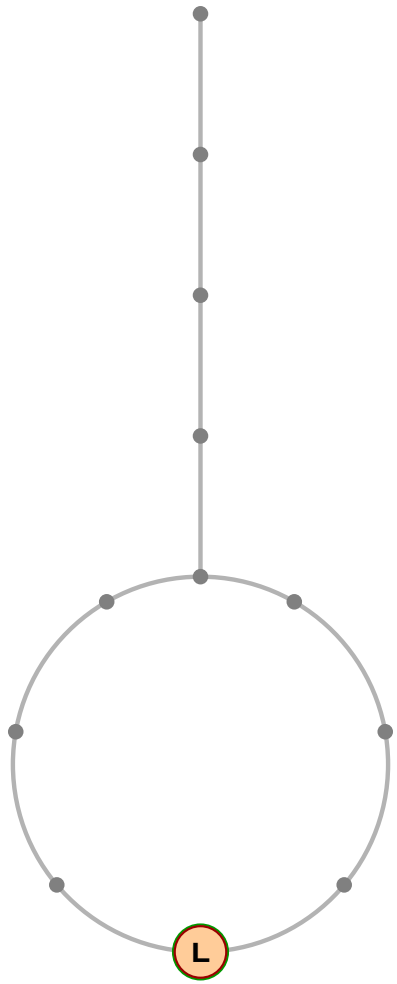
Étape 3 : Le lièvre entre dans le cycle

Algorithme de Floyd (Tortue / Lièvre)



Étape 4 : La tortue entre dans le cycle

Algorithme de Floyd (Tortue / Lièvre)



Étape 8 : Collision ! $x_8 = x_{16}$

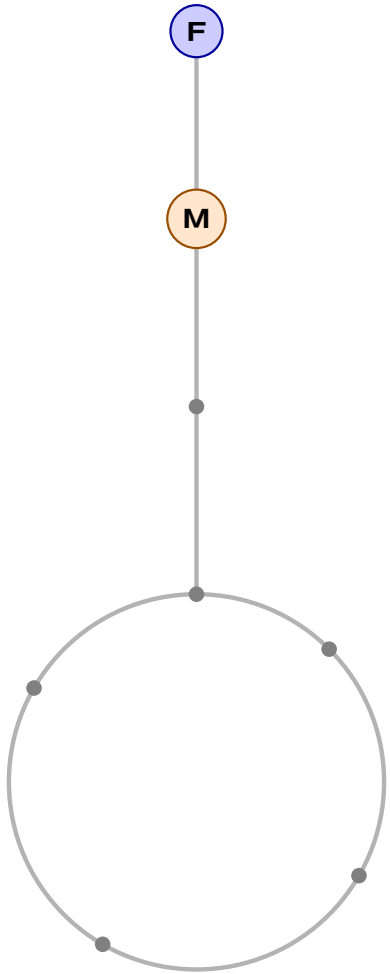
L'Algorithme de Brent

Idée : Au lieu de deux pointeurs mobiles, Brent utilise un point fixe x_{fixe} mis à jour uniquement aux puissances de 2 $(2^0, 2^1, 2^2, \dots)$.

1. **1 seule évaluation de** f par itération (contre 3).
2. Détection plus rapide du cycle dès que la fenêtre $2^k > L$.
3. Espérence de la detection de cycle en 1.98 $\sqrt{q}$

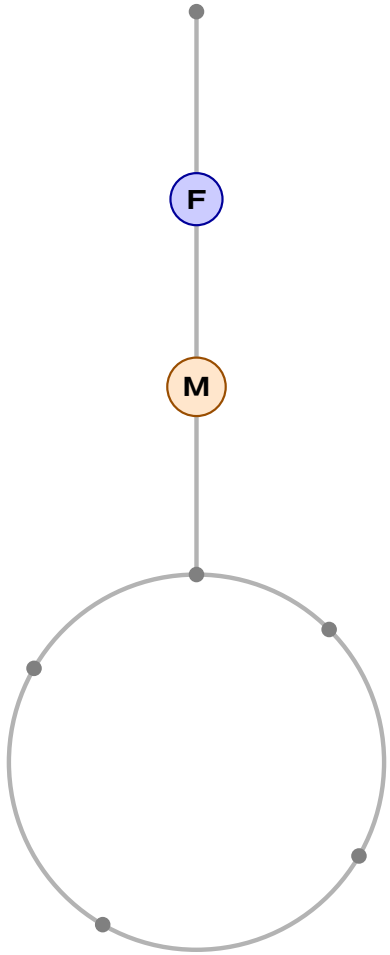
Gain estimé : Réduction de **36%** du nombre d'appels à f.

Algorithme de Brent (Fenêtrage)



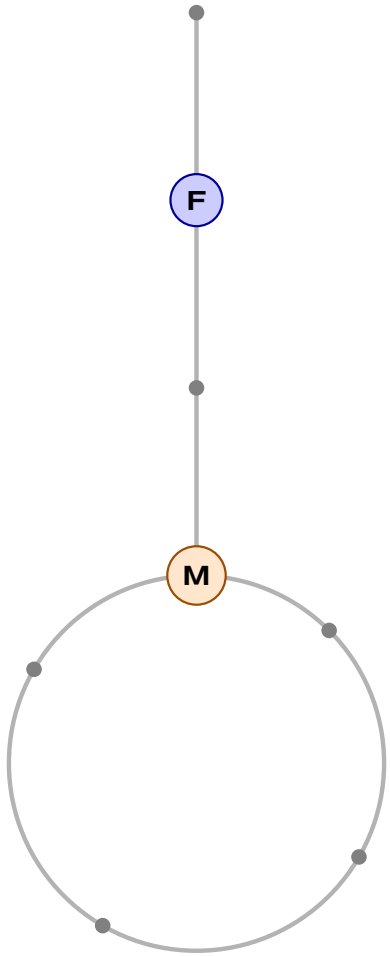
Fenêtre $2^0 = 1$: $x_{fixe} = x_0$, Test x_1

Algorithme de Brent (Fenêtrage)



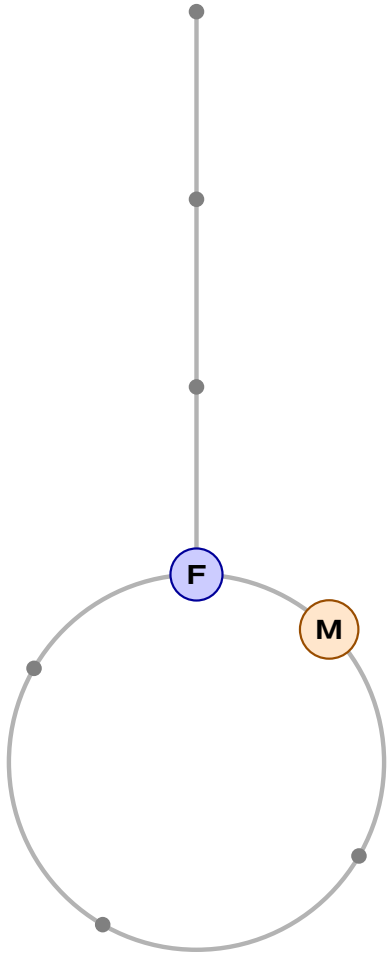
Fenêtre $2^1 = 2$: x_{fixe} se déplace en x_1, Test x_2

Algorithme de Brent (Fenêtrage)



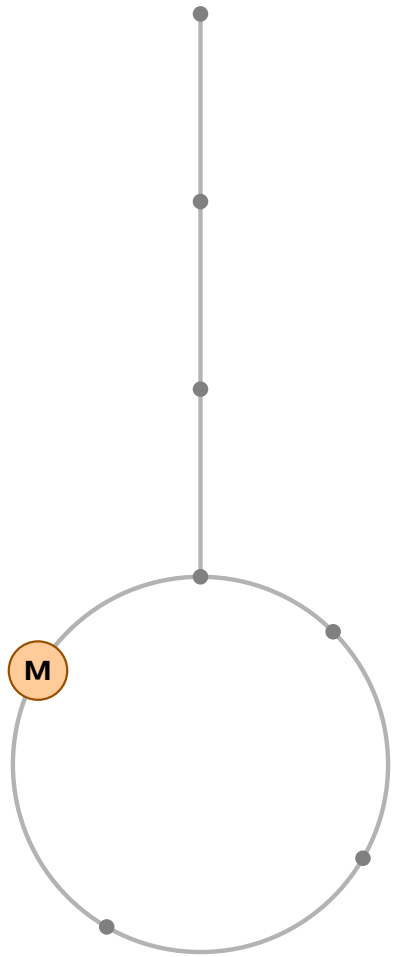
Fenêtre $2^1 = 2$: x_{fixe} reste, Test x_3

Algorithme de Brent (Fenêtrage)



Fenêtre $2^2 = 4$: x_{fixe} se déplace en x_3, Test x_4, x_5, x_6, x_7

Algorithme de Brent (Fenêtrage)



Collision ! Le point mobile a rattrapé la balise fixe

Partitionnement du Groupe

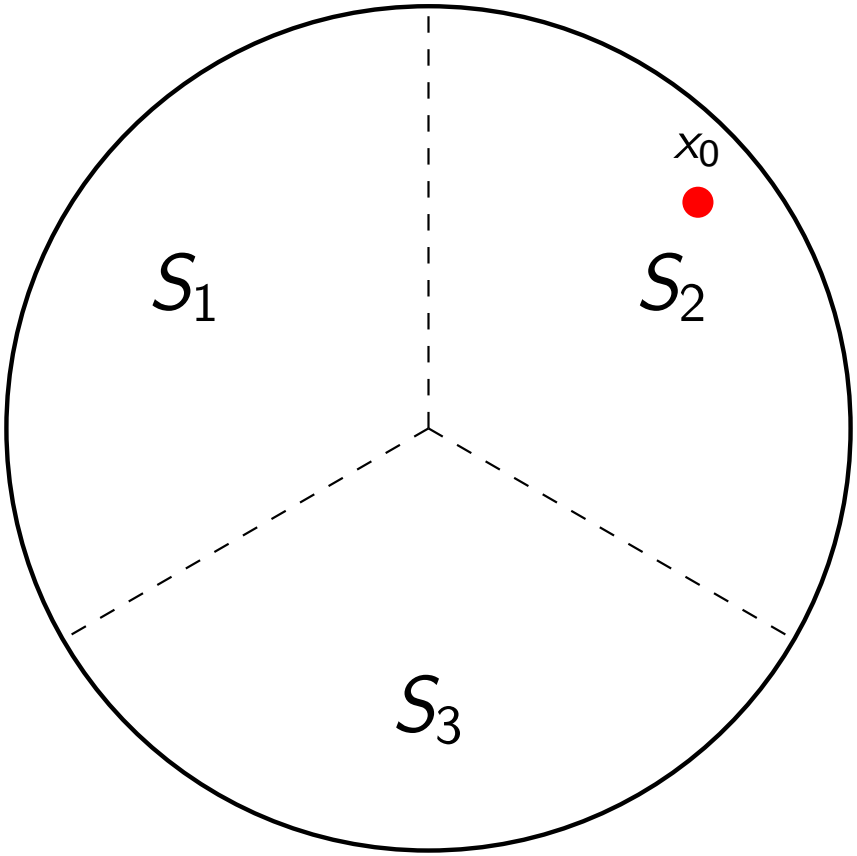
On simule le hasard en divisant G en 3 zones S_1, S_2, S_3.

$$x_{i+1} = \begin{cases} x_i^2 & \text{si } x \in S_1 \\ x_i \cdot g & \text{si } x \in S_2 \\ x_i \cdot h & \text{si } x \in S_3 \end{cases}$$

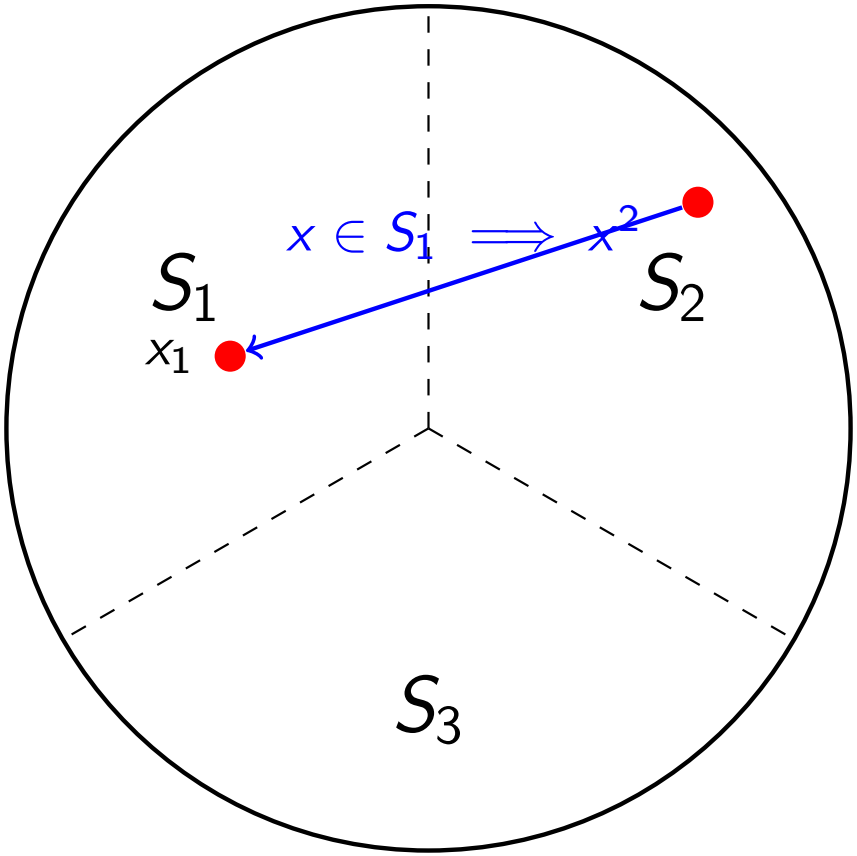
Analogie

C'est un mélangeur arithmétique qui brise la structure prévisible du groupe.

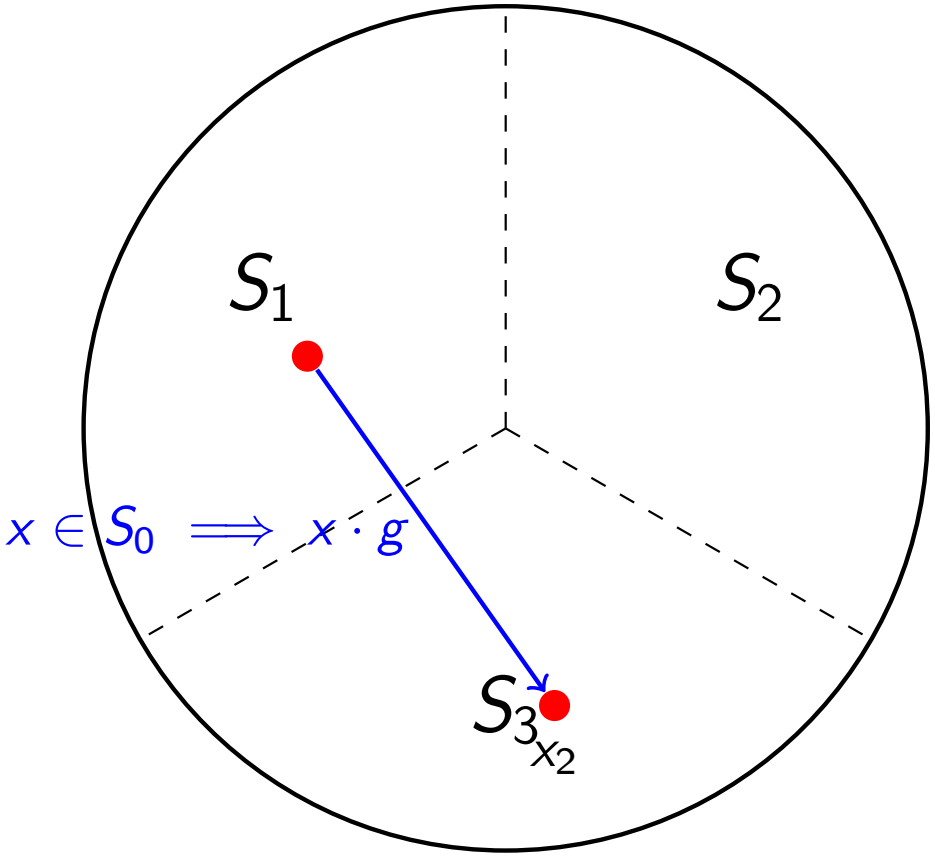
Partitionnement de Teske



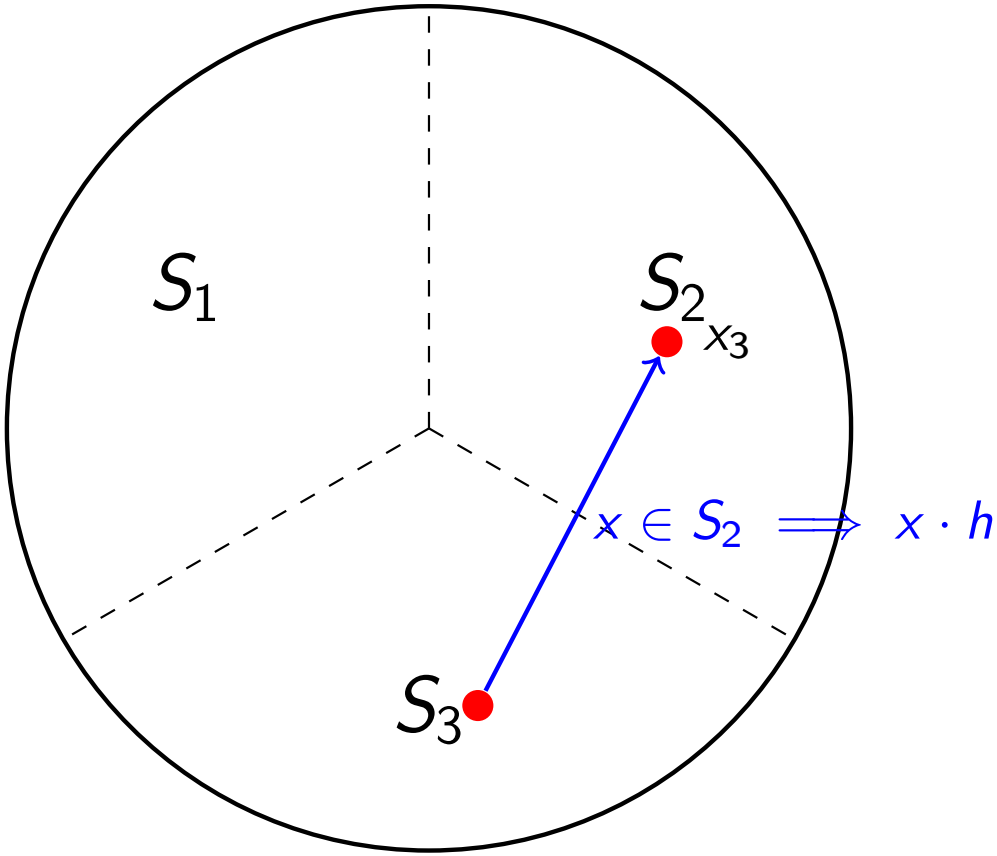
Partitionnement de Teske



Partitionnement de Teske



Partitionnement de Teske



Invariant et Suivi des Exposants

On maintient à chaque pas : $x_i = g^{a_i} h^{b_i} \pmod{p}$.

- Si on multiplie par g, alors $a \leftarrow a + 1$.
- Si on élève au carré, alors $a \leftarrow 2a$ et $b \leftarrow 2b$.

Les calculs sur (a, b) se font **modulo** q.

Structure explicite de la partition (Groupe $p = 13$)

Répartition statique du groupe $(\mathbb{Z}/13\mathbb{Z})^*$

On trie chaque élément $x \in \{1, \ldots, 12\}$ selon son reste modulo 3.

S_1 $(x \equiv 1)$

$$\{1, 4, 7, 10\}$$

Action : $x \leftarrow x \cdot g$
(Incrément de a)

S_2 $(x \equiv 2)$

$$\{2, 5, 8, 11\}$$

Action : $x \leftarrow x^2$
(Doublage a, b)

S_0 $(x \equiv 0)$

$$\{3, 6, 9, 12\}$$

Action : $x \leftarrow x \cdot h$
(Incrément de b)

De la Collision à la Solution

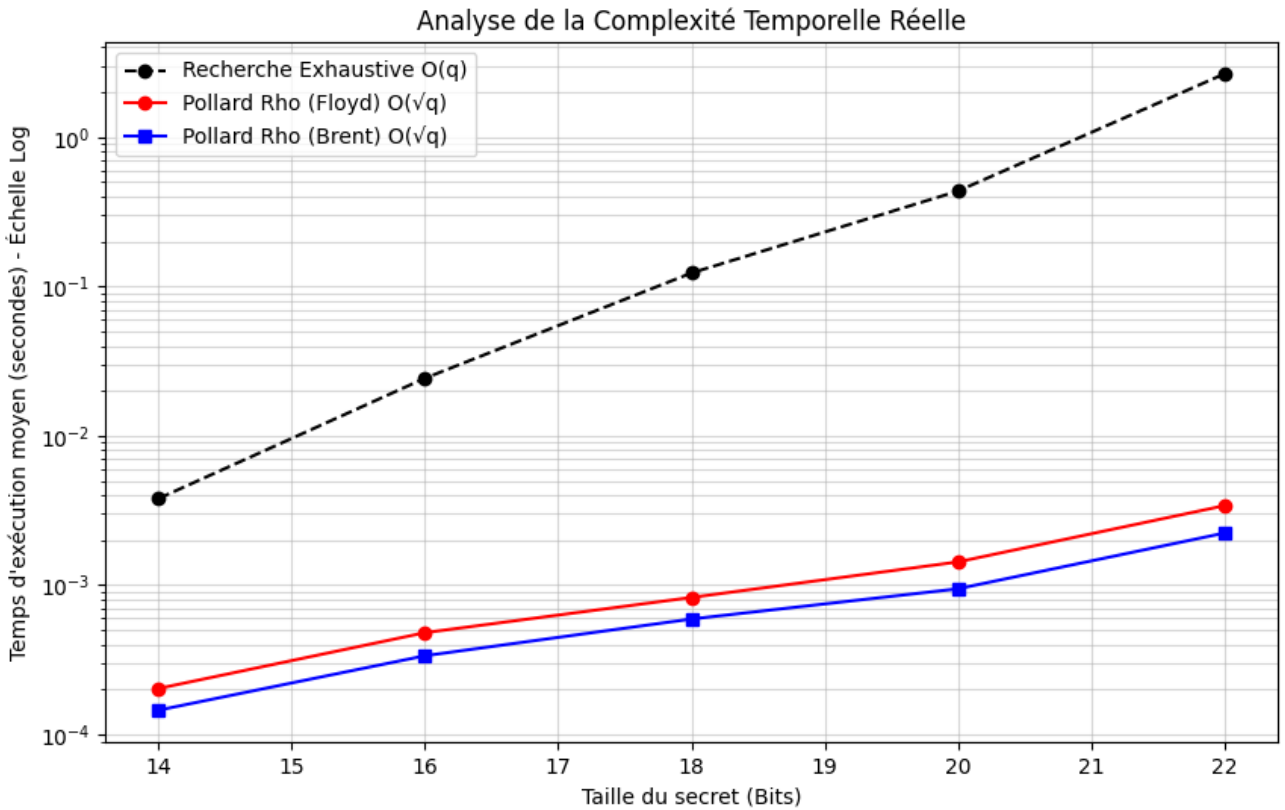
Si $x_i = x_j$, alors $g^{a_i} h^{b_i} \equiv g^{a_j} h^{b_j} \pmod{p}$.

$$\boxed{(b_j - b_i)x \equiv a_i - a_j \pmod{q}}$$

Exemple d'extraction

Soit $3x \equiv 6 \pmod{10}$.
$PGCD(3, 10) = 1 \implies x = 6 \cdot 3^{-1} \pmod{10}$.
$x \equiv 2 \pmod{10}$.

Analyse de la Complexité Temporelle



La pente double de l'Exhaustif confirme la rupture $O(q) \to O(\sqrt{q})$.

Vérification des statistiques de Flajolet

```
Statistiques de Flajolet (12 bits)
Moyenne Queue : Exp = 12.00 / Th = 21.53
Moyenne Cycle : Exp = 18.00 / Th = 21.53
Conformité : 39.36%
```

Validation Statistique et Biais

Mes résultats montrent une conformité de **40%** avec Flajolet.

- **Théorie :** La marche à 3 zones de Teske est biaisée.
- **Conséquence :** Elle est 20% plus longue qu'une marche idéale.

Ce "faible" résultat met en évidence les limites réelles de la partition à 3 zones.

Rupture de Diffie-Hellman (24 bits)

Scénario : Interception d'une clé DH

Soit un échange sur 24 bits : $g = 2$, $p = 14\,923\,061$, $h = 4\,579\,539$.
L'attaquant doit retrouver x tel que $2^x \equiv h \pmod{p}$.

```
Résultats :
  Force Brute : 8,928,617 itérations | 18.4320s
  Pollard Rho : 6,459 itérations | 0.0043s
  Facteur d'accélération : x4328
  Réduction des calculs  : x1382
  Solution trouvée       : x = 8928616
```

Leçon pour les Standards

Pollard Rho transforme un problème "humainement perceptible" en un calcul instantané. Pour une sécurité de 128 bits, il est **mathématiquement obligatoire** d'utiliser une clé de 256 bits.

Conclusion et Normes ANSSI

- Pollard Rho divise par deux la sécurité effective (en bits).
- **Diffie Hellman :** Clé 256 bits $\Longrightarrow$ 128 bits de sécurité.

Merci de votre attention !

Programme python I

```python
import math
import random
import time
import matplotlib.pyplot as plt


def algorithme_euclide_etendu(a, b):
    """Calcule le PGCD et les coefficients de Bézout."""
    if a == 0:
        return b, 0, 1
    pgcd, x1, y1 = algorithme_euclide_etendu(b % a, a)
    x = y1 - (b // a) * x1
    y = x1
    return pgcd, x, y

def inverse_modulaire(a, m):
    """Calcule l'inverse de a modulo m."""
    pgcd, x, y = algorithme_euclide_etendu(a, m)
    if pgcd != 1:
        return None
    return x % m

def resoudre_congruence_lineaire(v, u, q):
    """Résout l'équation v * x = u (mod q)."""
    pgcd, x0, y0 = algorithme_euclide_etendu(v, q)
    if u % pgcd != 0:
        return []

    solutions = []
    base_sol = (x0 * (u // pgcd)) % (q // pgcd)
    for i in range(pgcd):
        solutions.append((base_sol + i * (q // pgcd)) % q)
    return solutions

def est_premier(n):
```

Programme python II

```
36      """Test de primalité"""
37      if n < 2: return False
38      if n in (2, 3): return True
39      if n % 2 == 0: return False
40      for i in range(3, int(math.sqrt(n)) + 1, 2):
41          if n % i == 0: return False
42      return True
43
44
45  def etape_teske(x, a, b, g, h, p, q):
46      """Réalise une étape de la marche de Teske."""
47      zone = x % 3
48
49      if zone == 0:
50          nouveau_x = (x * x) % p
51          nouveau_a = (a * 2) % q
52          nouveau_b = (b * 2) % q
53      elif zone == 1:
54          nouveau_x = (x * g) % p
55          nouveau_a = (a + 1) % q
56          nouveau_b = b % q
57      else:
58          nouveau_x = (x * h) % p
59          nouveau_a = a % q
60          nouveau_b = (b + 1) % q
61
62      return nouveau_x, nouveau_a, nouveau_b
63
64  # Résolution du DLP
65
66  def resoudre_dlp_exhaustif(g, h, p, q):
67
68      nb_evaluations = 0
69      for x_candidat in range(q):
70          nb_evaluations += 1
```

Programme python III

```
71          if pow(g, x_candidat, p) == h:
72              return x_candidat, nb_evaluations
73      return None, nb_evaluations
74
75  def resoudre_dlp_floyd(g, h, p, q):
76
77      xt, at, bt = 1, 0, 0
78      xl, al, bl = 1, 0, 0
79
80      nb_evaluations = 0
81
82      while True:
83          xt, at, bt = etape_teske(xt, at, bt, g, h, p, q)
84          xl, al, bl = etape_teske(xl, al, bl, g, h, p, q)
85          xl, al, bl = etape_teske(xl, al, bl, g, h, p, q)
86
87          nb_evaluations += 3
88
89          if xt == xl:
90              v = (bl - bt) % q
91              u = (at - al) % q
92              solutions = resoudre_congruence_lineaire(v, u, q)
93              for x_cand in solutions:
94                  if pow(g, x_cand, p) == h:
95                      return x_cand, nb_evaluations
96
97  def resoudre_dlp_brent(g, h, p, q):
98
99      x_fixe, a_fixe, b_fixe = 1, 0, 0
00      x_actu, a_actu, b_actu = 1, 0, 0
01
02      puissance = 1
03      nb_evaluations = 0
04
05      while True:
```

Programme python IV

```
06          x_fixe, a_fixe, b_fixe = x_actu, a_actu, b_actu
07
08          for _ in range(puissance):
09              x_actu, a_actu, b_actu = etape_teske(x_actu, a_actu, b_actu, g, h, p, q)
10              nb_evaluations += 1
11
12              if x_actu == x_fixe:
13                  v = (b_actu - b_fixe) % q
14                  u = (a_fixe - a_actu) % q
15                  solutions = resoudre_congruence_lineaire(v, u, q)
16                  for x_cand in solutions:
17                      if pow(g, x_cand, p) == h:
18                          return x_cand, nb_evaluations
19
20          puissance *= 2
21
22  # Statistiques et Benchmark
23
24  def verifier_statistiques_flajolet(nb_bits, nb_essais=5000):
25      """Vérification des structures de Rho selon Flajolet et Odlyzko."""
26      print(f"Statistiques de Flajolet ({nb_bits} bits)")
27
28      p = 0
29      while not est_premier(p) or p < 100:
30          p = random.getrandbits(nb_bits)
31      q = p - 1
32      g = 2
33      h = pow(g, random.randint(2, q-1), p)
34
35      queues = []
36      cycles = []
37
38      for _ in range(nb_essais):
39          historique = {}
40          x, a, b = 1, 0, 0
```

Programme python V

```
41          i = 0
42          while x not in historique:
43              historique[x] = i
44              x, a, b = etape_teske(x, a, b, g, h, p, q)
45              i += 1
46
47          mu = historique[x]
48          lam = i - historique[x]
49          queues.append(mu)
50          cycles.append(lam)
51
52      avg_mu = sum(queues) / nb_essais
53      avg_lam = sum(cycles) / nb_essais
54      theorie = math.sqrt((math.pi * q) / 8)
55
56      print(f"Moyenne Queue : Exp = {avg_mu:.2f} / Th = {theorie:.2f}")
57      print(f"Moyenne Cycle : Exp = {avg_lam:.2f} / Th = {theorie:.2f}")
58      ecart = (abs(avg_mu + avg_lam - 2*theorie) / theorie) * 100
59      print(f"Conformité : {100 - ecart:.2f}%\n")
60
61  def executer_comparaison_performance(essais=15):
62      """Benchmark du nombre d'évaluations entre Floyd et Brent."""
63      tailles_bits = [14, 16, 18, 20, 22]
64
65      stats_floyd = []
66      stats_brent = []
67      gains_pourcentage = []
68
69      print("Comparaison du nombre d'évaluations")
70      print(f"{'Bits':<6} {'Floyd':<12} {'Brent':<12} {'Gain (%)':<10}")
71      print("-" * 42)
72
73      for b in tailles_bits:
74          total_eval_f = 0
75          total_eval_b = 0
```

Programme python VI

```
76
77          for _ in range(essais):
78              p = 0
79              while not est_premier(p):
80                  p = random.getrandbits(b)
81              q = p - 1
82              g = random.randint(2, p-2)
83              secret = random.randint(2, q-1)
84              h = pow(g, secret, p)
85
86              _, ev_f = resoudre_dlp_floyd(g, h, p, q)
87              total_eval_f += ev_f
88
89              _, ev_b = resoudre_dlp_brent(g, h, p, q)
90              total_eval_b += ev_b
91
92          moy_f = total_eval_f / essais
93          moy_b = total_eval_b / essais
94          gain = ((moy_f - moy_b) / moy_f) * 100
95
96          stats_floyd.append(moy_f)
97          stats_brent.append(moy_b)
98          gains_pourcentage.append(gain)
99
00          print(f"{b:<6} {moy_f:<12.1f} {moy_b:<12.1f} {gain:.2f}%")
01
02      print("-" * 42)
03      gain_moyen_total = sum(gains_pourcentage) / len(gains_pourcentage)
04      print(f"Gain moyen de Brent : {gain_moyen_total:.2f}%\n")
05
06      plt.figure(figsize=(10, 6))
07      plt.plot(tailles_bits, stats_floyd, 'ro-', label='Floyd')
08      plt.plot(tailles_bits, stats_brent, 'bs-', label='Brent')
09      plt.yscale('log')
10      plt.xlabel('Taille du groupe (Bits)')
```

Programme python VII

```
    plt.ylabel('  valuations   (log)')
    plt.title('Comparaison d\'efficacité : Floyd vs Brent')
    plt.legend()
    plt.grid(True, which="both", ls="-", alpha=0.5)
    plt.text(tailles_bits[1], stats_floyd[-1], f"Gain moyen : {gain_moyen_total:.1f}%",
             bbox=dict(facecolor='green', alpha=0.2), fontsize=12)
    plt.show()

def executer_benchmark_temporel(essais=15):
    """Analyse du temps d'exécution réel (CPU Time)."""
    tailles_bits = [14, 16, 18, 20, 22]

    temps_exh = []
    temps_floyd = []
    temps_brent = []

    print("Benchmark temporel")
    print(f"{'Bits':<6} {'Exhaustif':<15} {'Floyd':<15} {'Brent':<15}")
    print("-" * 55)

    for b in tailles_bits:
        t_exh, t_floyd, t_brent = 0.0, 0.0, 0.0
        for _ in range(essais):
            p = 0
            while not est_premier(p): p = random.getrandbits(b)
            q = p - 1
            g = random.randint(2, p-2)
            h = pow(g, random.randint(2, q-1), p)

            start = time.perf_counter()
            resoudre_dlp_exhaustif(g, h, p, q)
            t_exh += (time.perf_counter() - start)

            start = time.perf_counter()
            resoudre_dlp_floyd(g, h, p, q)
```

Programme python VIII

```
46              t_floyd += (time.perf_counter() - start)
47
48              start = time.perf_counter()
49              resoudre_dlp_brent(g, h, p, q)
50              t_brent += (time.perf_counter() - start)
51
52          moy_e, moy_f, moy_b = t_exh/essais, t_floyd/essais, t_brent/essais
53          temps_exh.append(moy_e)
54          temps_floyd.append(moy_f)
55          temps_brent.append(moy_b)
56
57          print(f"{b:<6} {moy_e:<15.6f} {moy_f:<15.6f} {moy_b:<15.6f}")
58      print()
59
60      plt.figure(figsize=(10, 6))
61      plt.plot(tailles_bits, temps_exh, 'ko--', label='Exhaustive O(q)')
62      plt.plot(tailles_bits, temps_floyd, 'ro-', label='Floyd O( q )')
63      plt.plot(tailles_bits, temps_brent, 'bs-', label='Brent O( q )')
64      plt.yscale('log')
65      plt.xlabel('Taille du secret (Bits)')
66      plt.ylabel('Temps moyen (s) - Log')
67      plt.title('Complexité Temporelle')
68      plt.legend()
69      plt.grid(True, which="both", ls="-", alpha=0.5)
70      plt.show()
71
72  def comparaison_finale_exhaustive_vs_pollard():
73      """Bilan final pour l'instance Diffie Hellman de 24 bits."""
74      print("Force Brute vs Pollard Rho :")
75
76      bits = 24
77      p = 0
78      while not est_premier(p):
79          p = random.randint(2**(bits-1), 2**bits - 1)
80      q = p - 1
```

Programme python IX

```
    g = 2
    secret = random.randint(2, q - 1)
    h = pow(g, secret, p)

    print(f"Instance : 2^x = {h} (mod {p})")

    start_exh = time.perf_counter()
    sol_exh, it_exh = resoudre_dlp_exhaustif(g, h, p, q)
    t_exh = time.perf_counter() - start_exh

    start_pollard = time.perf_counter()
    sol_p, it_pollard = resoudre_dlp_brent(g, h, p, q)
    t_pollard = time.perf_counter() - start_pollard

    facteur_vitesse = t_exh / t_pollard if t_pollard > 0 else 0
    reduction_operations = it_exh / it_pollard

    print("\nRésultats :")
    print(f"  Force Brute : {it_exh:,} itérations | {t_exh:.4f}s")
    print(f"  Pollard Rho : {it_pollard:,} itérations | {t_pollard:.4f}s")
    print(f"  Facteur d'accélération : x{int(facteur_vitesse)}")
    print(f"  Réduction des calculs  : x{int(reduction_operations)}")
    print(f"  Solution trouvée       : x = {int(sol_p)}")

if __name__ == "__main__":

    verifier_statistiques_flajolet(12, nb_essais=300)
    executer_comparaison_performance(300)
    executer_benchmark_temporel()
    comparaison_finale_exhaustive_vs_pollard()
```